1 ОБЗОР ЛИТЕРАТУРЫ

1.1 Цели и задачи проекта

Для начала разработки программного средства необходимо определить цели и задачи проекта. Целью дипломного проекта является разработка программного средства для организации и управления волонтёрской деятельностью, представляющего собой клиент-серверное мобильное приложение для автоматизации работы с мероприятиями и заявками, информирования участников, координации волонтёров в процессе проведения мероприятий, а также формирования системы оценки их надёжности.

Для достижения поставленной цели необходимо решить следующие задачи:

– регистрация, аутентификация и авторизация пользователей;

– хранение и обработка данных о пользователях, мероприятиях и заявках;

– просмотр перечня доступных волонтерских мероприятий и получение подробной информации о них;

– просмотр и редактирование персональных данных пользователя;

– создание, редактирование и удаление мероприятий;

– отображение текущего статуса мероприятий;

– подача заявок на участие в мероприятиях и их последующая обработка;

– распределение волонтеров по группам во время проведения мероприятия;

– оперативное управление волонтерами в процессе проведения мероприятия;

– информирование пользователей о событиях и изменениях в системе;

– формирование рейтинга надежности волонтеров.

1.2 Выдвигаемые требования к проекту

На основании поставленной цели и сформулированных задач следует определить функциональные и нефункциональные требования к разрабатываемому программному средству.

1.2.1 Функциональные требования

Функциональные требования определяют перечень функций, которые должно реализовывать программное средство для обеспечения выполнения поставленных задач.

Разрабатываемое программное средство должно обеспечивать выполнение следующих функциональных требований:

1 Регистрация, аутентификация и авторизация пользователей. Приложение должно обеспечивать возможность создания учетной записи, входа в систему и разграничения прав доступа в зависимости от роли

пользователя. В системе должны быть предусмотрены различные категории пользователей, в том числе волонтеры и организации.

2 Просмотр и редактирование персональных данных пользователя. Пользователь должен иметь возможность просматривать и изменять данные своего профиля, необходимые для работы в системе. К таким данным относятся имя, номер телефона и иные персональные данные.

3 Хранение и обработка данных о пользователях, мероприятиях и заявках. Система должна обеспечивать сохранение, обновление, удаление и получение информации, необходимой для функционирования программного средства, что включает в себя данные о зарегистрированных пользователях, создаваемых мероприятиях, а также поданных заявках на участие.

4 Просмотр перечня доступных волонтерских мероприятий и получение подробной информации о них. Приложение должно предоставлять пользователю список доступных мероприятий с возможностью получения полной информации о каждом из них. В описании мероприятия должны содержаться его основные характеристики, включая наименование, описание, дату и время проведения, место, категорию и иные сведения. Пользователь должен иметь возможность просматривать мероприятия на карте, а также выполнять поиск и фильтрацию мероприятий по месту, категории, времени проведения и другим параметрам.

5 Создание, редактирование и удаление мероприятий. Для организатора и организаций предусмотрены функции добавления новых мероприятий, изменения их параметров и удаления. Администратор осуществляет модерацию добавляемых мероприятий: рассматривает поступившую информацию, принимает решение о публикации либо отклоняет мероприятие с указанием причины.

6 Подача заявок на участие в мероприятиях и их последующая обработка. В приложении должна быть реализована функция отправки заявок на участие в выбранных мероприятиях. Организатор рассматривает поступившие заявки, принимает решение об их подтверждении или отклонении, а также при необходимости изменяет текущий статус заявки.

7 Распределение волонтеров по группам во время проведения мероприятия. Система должна обеспечивать возможность распределения принятых волонтёров по группам после начала мероприятия. Организатор может назначать лидеров групп, формировать состав групп, изменять распределение участников и при необходимости корректировать роли волонтёров в процессе проведения мероприятия.

8 Оперативное управление волонтерами в процессе проведения мероприятия. В приложении должны быть реализованы средства координации участников во время активного мероприятия. Организатор и лидеры групп могут взаимодействовать с волонтёрами через чаты, изменять состав групп и передавать актуальную информацию, необходимую для выполнения мероприятия.

9 Информирование пользователей о событиях и изменениях в системе. В приложении должна быть реализована система уведомлений,

информирующая пользователей о важных изменениях: подтверждении или отклонении заявки, начале мероприятия и других значимых действиях организатора.

10 Формирование рейтинга надежности волонтеров. Система должна формировать рейтинг надёжности волонтёров на основе их участия в мероприятиях, отмены заявок, оценок со стороны организаторов и лидеров групп. В профиле волонтёра должна отображаться история изменения рейтинга с указанием события и причины начисления или списания баллов.

1.2.2 Нефункциональные требования

Нефункциональные требования характеризуют свойства и качественные параметры программного средства, которые должны обеспечиваться в процессе его работы.

К разрабатываемому программному средству предъявляются следующие нефункциональные требования:

1 Производительность. При выполнении основных операций, таких как вход в систему, загрузка мероприятий, подача заявок и обновление данных, программное средство должно обеспечивать приемлемую скорость отклика. Кроме того важно, чтобы обмен данными между мобильным приложением, серверной частью и базой данных выполнялся без значительных задержек, так как это напрямую влияет на эффективность использования системы.

2 Надежность. Обеспечение надежности необходимо для корректного функционирования приложения в процессе его практического использования. Поскольку в системе выполняется работа с мероприятиями, заявками и данными пользователей, важно, чтобы приложение устойчиво функционировало при возникновении ошибок, временных сбоев соединения или некорректного ввода данных.

3 Совместимость. Особое значение для разрабатываемого приложения имеет корректная работа на мобильных устройствах под управлением операционной системы (ОС) iOS. При этом необходимо учитывать различия в размерах экранов, особенностях отображения интерфейса и версии операционной системы. Интерфейс приложения должен сохранять целостность и корректность отображения на различных моделях мобильных устройств.

4 Защита данных. Необходимо обеспечить защиту персональных данных пользователей и служебной информации системы. Это включает безопасную аутентификацию, разграничение прав доступа для пользователей, а также защищенную передачу данных между клиентской и серверной частями. Реализация данного требования необходима для предотвращения несанкционированного доступа и обеспечения корректной работы системы с различными категориями пользователей.

5 Масштабируемость. Структура программного средства должна обеспечивать возможность дальнейшего расширения его функциональности без существенного изменения уже реализованных компонентов системы. Это особенно важно для приложения, так как в дальнейшем могут быть добавлены

дополнительные функции управления мероприятиями.

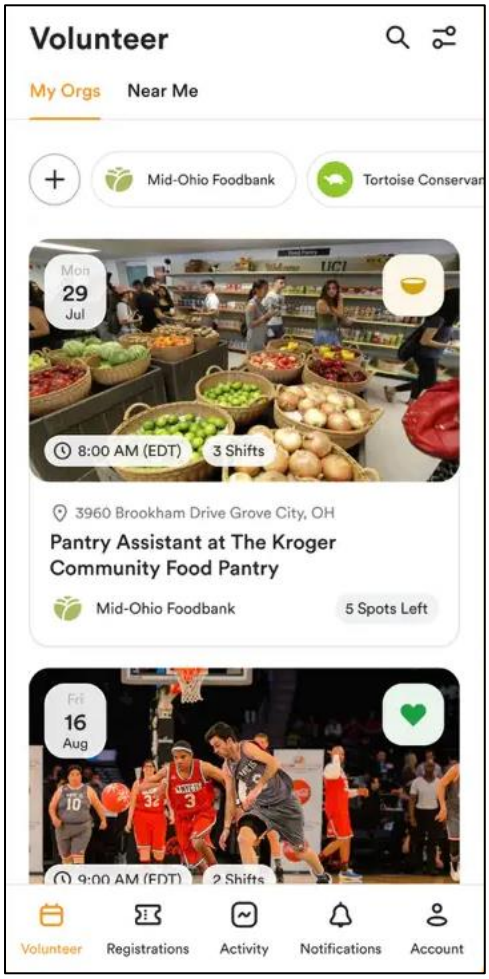
1.3 Обзор аналогов

Перед разработкой мобильного приложения необходимо провести анализ существующих аналогов, представленных на рынке. Такой обзор позволяет изучить уже реализованные подходы к решению схожих задач, определить наиболее востребованные функции, оценить удобство пользовательского интерфейса и выявить сильные и слабые стороны существующих решений.

1.3.1 Платформа POINT-Volunteer near

POINT-Volunteer near you представляет собой мобильное iOS-приложение, предназначенное для поиска доступных волонтерских мероприятий, регистрации на мероприятия и взаимодействия пользователей с организациями. Пользователь может находить подходящие события, записываться на участие и отслеживать собственную активность [1]. Интерфейс приложения представлен на рисунке 1.1.

а)



б)

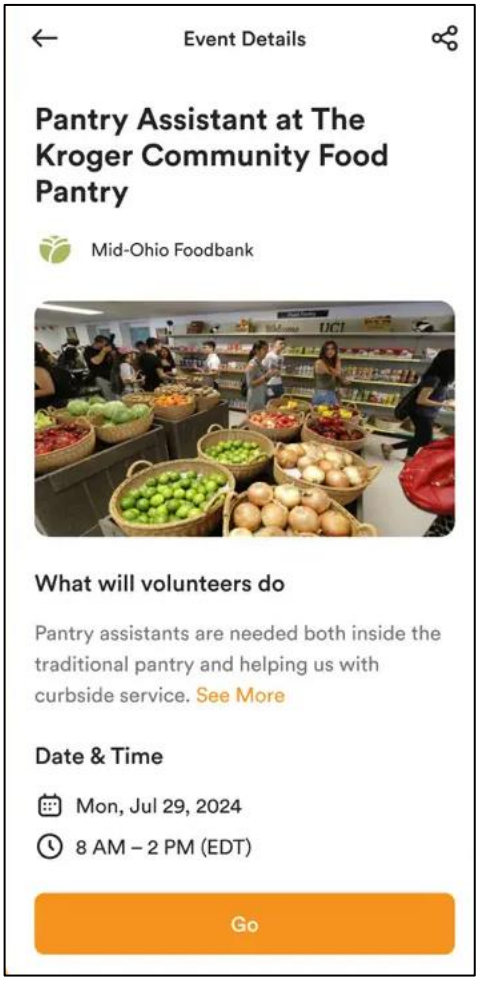


Рисунок 1.1 – Интерфейс приложения POINT:
а – главное меню; *б* – карточка мероприятия и запись на участие

На рисунке 1.1, а показан главный экран приложения с лентой доступных мероприятий. В верхней части располагаются вкладки, позволяющие переключаться между событиями выбранных организаций и ближайшими мероприятиями. Также доступны значки поиска и фильтрации, что повышает удобство навигации. Ниже размещены карточки мероприятий, содержащие подробную информацию. Дополнительно внизу расположена навигационная панель, которая делает перемещение по приложению интуитивно понятным. На рисунке 1.1, б показан экран подробной информации о мероприятии. Здесь отображаются название события, организация, иллюстрация, описание предстоящей деятельности, дата и время проведения. Интерфейс выглядит визуально аккуратно, а крупные элементы делают взаимодействие удобным. В то же время можно отметить, что подпись кнопки Go является слишком общей и не всегда однозначно отражает конкретное действие, например запись на мероприятие.

К основным преимуществам POINT-Volunteer near you относятся:

– быстрое нахождение волонтерские мероприятия по различным направлениям деятельности;

– предоставление пользователю персонализированной ленты событий с учетом его интересов и выбранных организаций;

– упрощение процесса регистрации на участие в волонтерских событиях;

– возможность отслеживания волонтерских часов и хранение истории активности пользователя.

К недостаткам POINT-Volunteer near you относятся:

– функциональность приложения в большей степени ориентирована на поиск и выбор мероприятий, чем на координацию действий волонтеров в процессе их проведения;

– возможности взаимодействия между волонтёрами и организаторами внутри самого мероприятия ограничены;

– наличие технических сбоев и нестабильной работы приложения, включая зависания, сбои при входе в учетную запись и проблемы с доступом к ранее сохранённым данным об активности пользователя;

– при возврате к списку возможных мероприятий результаты поиска могут сбрасываться, из-за чего параметры приходится вводить повторно.

Таким образом, несмотря на удобство поиска волонтерских мероприятий, приложение имеет ряд недостатков, связанных со стабильностью работы, удобством использования и ограниченными возможностями сопровождения волонтерской деятельности.

1.3.2 Приложение VicNet

VicNet («Сеть волонтеров») – это мобильное iOS-приложение, входящее в состав системы Volgistics и предназначенное для информационного сопровождения волонтеров в рамках конкретной организации. Его основное назначение заключается в предоставлении пользователю доступа к личному расписанию, данным о назначенных сменах, истории отработанных часов,

уведомлениям и персональной информации [2]. Функциональные возможности приложения представлены на рисунке 1.2. В отличие от приложений, ориентированных на поиск доступных волонтерских мероприятий, VicNet используется преимущественно как внутренний инструмент учета и координации уже зарегистрированных участников волонтерской деятельности.

а) б)

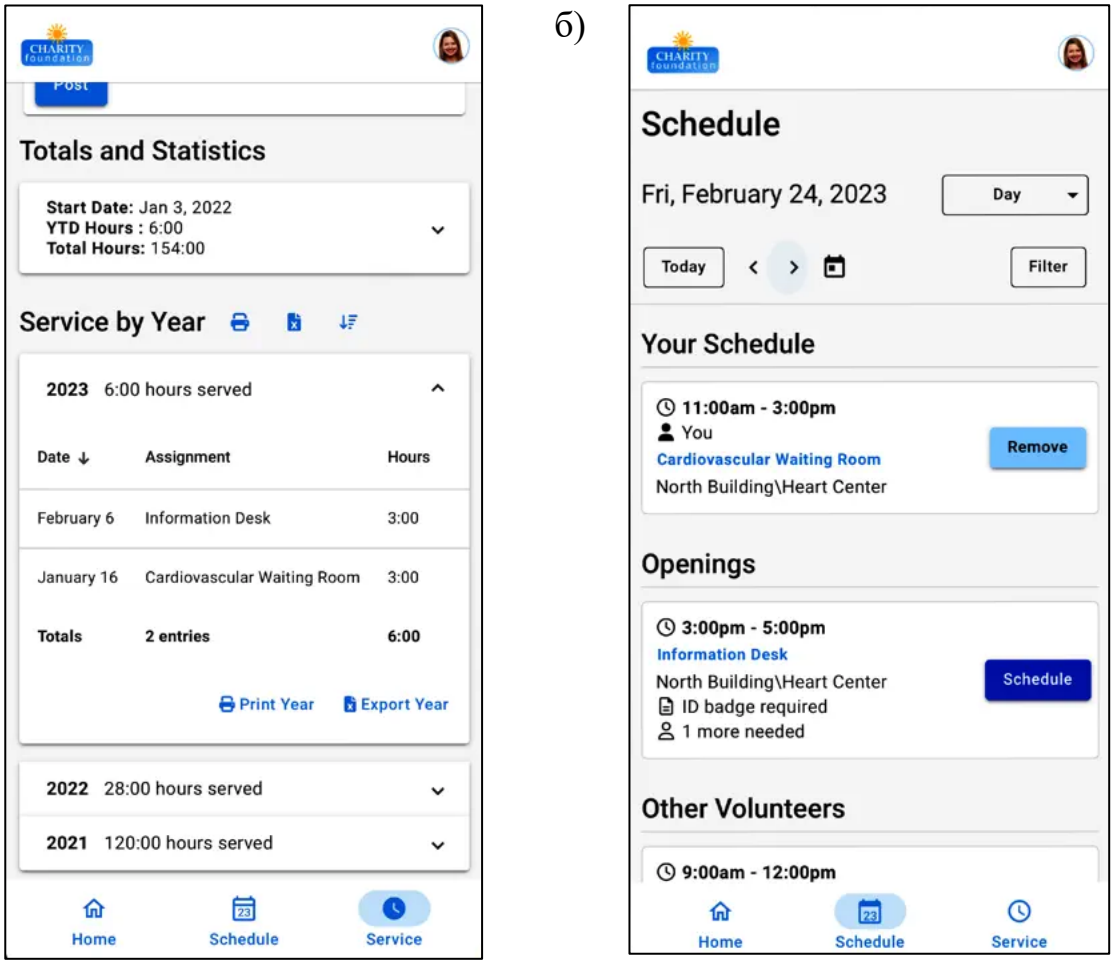


Рисунок 1.2 – Интерфейс приложения VicNet:
а – учет активности; *б* – управление расписанием

На рисунке 1.2, а представлен интерфейс просмотра статистики и истории волонтерской деятельности. Пользователю доступны данные о дате начала участия, количестве часов за текущий период и общем числе отработанных часов. Ниже представлен раздел Service by Year, в котором история активности сгруппирована по годам. Для каждого года отображаются выполненные назначения, даты, наименования мест службы и количество часов. Также предусмотрены функции печати и экспорта данных за выбранный период. Такой экран удобен тем, что позволяет быстро оценить накопленную статистику и просмотреть историю участия.

На рисунке 1.2, б представлен экран расписания. На нем пользователь может просматривать текущую дату, переключаться между днями,

использовать фильтрацию и видеть свое расписание. Отдельно выделяются блоки Your Schedule, Openings и Other Volunteers, что позволяет различать уже назначенные смены, доступные для записи интервалы и сведения о других участниках. Пользователь может удалить выбранную смену либо записаться на свободное время. В нижней части экрана размещена панель навигации, обеспечивающая переход между основными разделами приложения: Home, Schedule и Service.

Преимущества VicNet:

– позволяет просматривать назначенные смены и управлять личным расписанием;

– дает возможность отслеживать историю волонтерской деятельности и количество отработанных часов;

– поддерживает получение сообщений и уведомлений от координаторов;

– позволяет обновлять персональные данные пользователя.

Недостатки VicNet:

– приложение полностью зависит от экосистемы Volgistics и не является самостоятельной платформой для поиска волонтерских возможностей;

– функциональность приложения в большей степени ориентирована на учет, расписание и сопровождение участия, чем на привлечение новых волонтеров.

VicNet является специализированным инструментом для организации работы волонтеров, уже включенных в деятельность конкретной организации. Приложение удобно использовать для просмотра расписания, учета часов и получения служебной информации, однако его зависимость от системы Volgistics и отсутствие функций самостоятельного поиска волонтерских возможностей ограничивают сферу применения данного решения.

1.3.3 Приложение HelperHelper

HelperHelper – мобильное iOS-приложение для организации волонтерской деятельности и учета активности участников. Оно позволяет волонтерам находить доступные мероприятия, регистрироваться на них, отслеживать свою активность и получать информацию о предстоящих событиях. Для координаторов приложение предоставляет средства управления мероприятиями, распределения участников и контроля их активности. Таким образом, приложение может использоваться как инструмент планирования участия в волонтерской деятельности и учета активности волонтеров [3].

Главный экран приложения и информация о мероприятии представлены на рисунке 1.3.

Преимуществами HelperHelper являются:

– наличие функций поиска доступных волонтерских мероприятий;

– возможность регистрации на участие в выбранных событиях;

– средства учета и отслеживания активности волонтеров;

– наличие инструментов для координаторов, связанных с управлением

мероприятиями и распределением участников.

Недостатками HelperHelper являются:

– сервис ориентирован прежде всего на учёт и организацию волонтёрской деятельности в целом, чем на организацию непосредственно во время проведения мероприятий;

– приложение поддерживает только английский язык.

а) б)

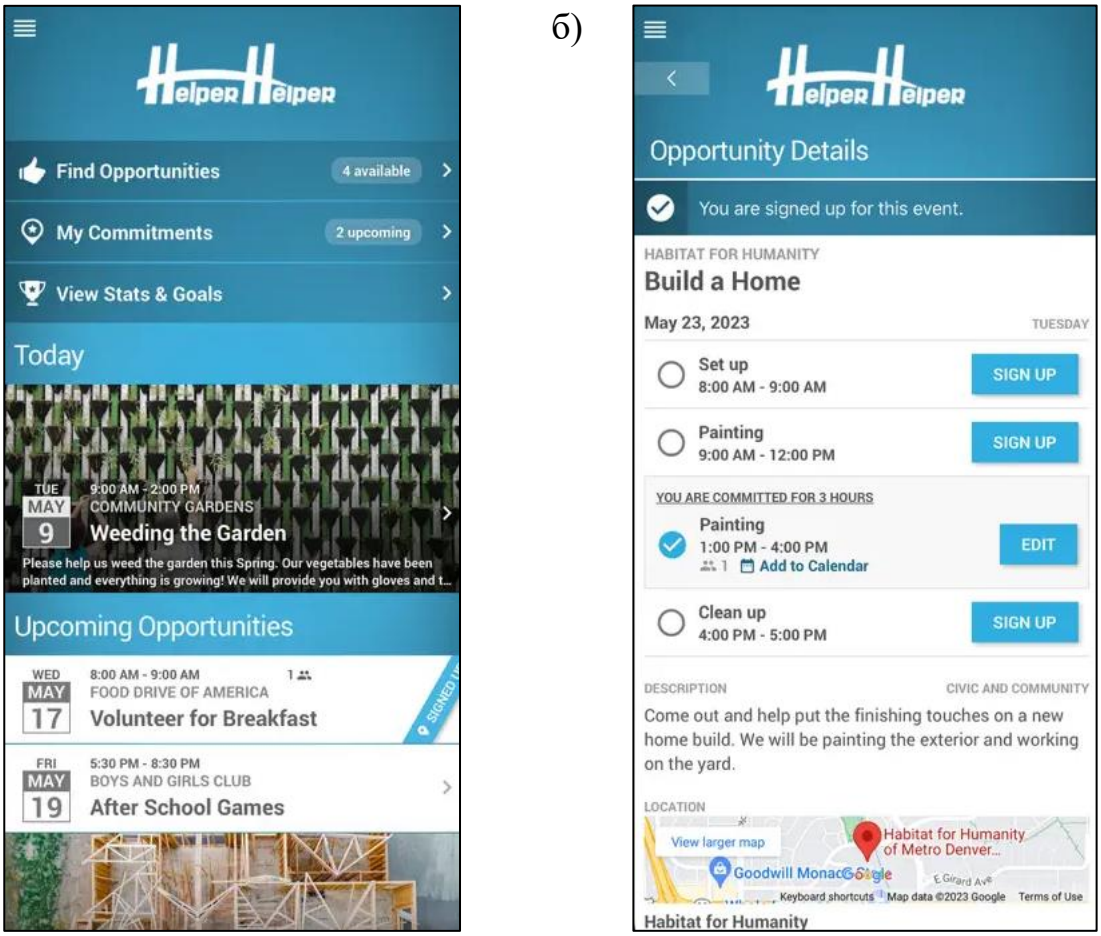


Рисунок 1.3 – Интерфейс приложения HelperHelper:
а – главный экран приложения; *б* – информация о мероприятии

На рисунке 1.3, а показан главный экран приложения. В верхней части размещены основные разделы. Ниже отображаются блоки Today и Upcoming Opportunities, в которых пользователь может просматривать актуальные и предстоящие волонтерские возможности. Карточки мероприятий содержат дату, время, название организации, название события и краткую информацию о статусе регистрации. Можно выделить, что из-за плотного размещения элементов и текстовой информации экран выглядит несколько перегруженным.

На рисунке 1.3, б отображается раздел подробной информации о мероприятии. Также пользователь может записаться на отдельную смену, изменить уже выбранное участие или добавить событие в календарь. Дополнительную информативность обеспечивает отображение

местоположения мероприятия и его картографического представления.

HelperHelper предоставляет удобные средства для регистрации на мероприятия и контроля вовлеченности участников, однако его функциональность в большей степени связана с административным сопровождением волонтерской деятельности, чем с поддержкой взаимодействия пользователей непосредственно в процессе проведения мероприятий.

Таким образом, среди рассмотренных аналогов нет программного средства, которое обеспечивало бы оперативное управление волонтерами в ходе проведения мероприятий и формирование рейтинга надежности участников. Для реализации таких функций требуется использование нескольких отдельных средств, что усложняет организацию работы и снижает удобство управления волонтерской деятельностью. Следовательно, разработка дипломного проекта является целесообразной, так как проектируемое программное средство должно объединить указанные функции в рамках одной системы.

1.4 Обзор языков программирования для разработки серверной части приложения

Для разработки серверной части клиент-серверного приложения могут использоваться различные языки программирования, отличающиеся подходом к организации вычислений, производительностью, сложностью сопровождения и удобством разработки. К наиболее подходящим из них относятся:

1 Java. Этот язык программирования обычно используется при создании крупных серверных и корпоративных приложений. Java обеспечивает переносимость приложений за счет принципа «write once, run anywhere», а платформа Java EE предназначена для разработки надежных, защищенных, переносимых и масштабируемых многоуровневых приложений [4]. К преимуществам Java можно отнести зрелую экосистему, высокий уровень надежности, развитые средства типизации и широкое применение в enterprise-разработке. К недостаткам обычно относят более высокий порог вхождения, больший объем шаблонного кода и более длительный цикл разработки.

2 Go. Ориентирован на создание производительных и масштабируемых сервисов. Язык создавался с учетом задач concurrency, cloud-разработки и микросервисной архитектуры [5]. Кроме того, Go-код компилируется в единый статический бинарный файл, что упрощает развертывание сервисов. К преимуществам Go можно отнести высокую скорость работы, удобную модель конкурентного выполнения и сравнительно простое развертывание. Недостатками являются более ограниченный набор высокоуровневых средств и более низкая скорость начальной разработки при решении типовых прикладных задач.

3 JavaScript используется для разработки серверной части, как правило, в среде Node.js. Node.js представляет собой асинхронную событийно-

ориентированную среду выполнения JavaScript, предназначенную для построения масштабируемых сетевых приложений [6]. Из преимуществ следует выделить неблокирующую модель ввода-вывода, удобство обработки большого количества соединений и использование одного языка как на клиентской, так и на серверной стороне. Вместе с тем применение JavaScript на сервере может сопровождаться усложнением логики при разработке крупных систем, а однопоточная модель по умолчанию требует аккуратного подхода к выполнению ресурсоемких операций.

4 Python широко используется при разработке серверной части благодаря лаконичному синтаксису, высокой читаемости кода и сравнительно быстрой реализации программных решений. К его преимуществам относятся удобство сопровождения, наличие большого количества библиотек и фреймворков [7]. В частности, для разработки серверной части на Python может использоваться фреймворк FastAPI, который позволяет структурированно организовать обработку запросов, работу с маршрутами и взаимодействие с клиентским приложением. Однако, при разработке отдельных компонентов может потребоваться дополнительная оптимизация, а также для некоторых задач могут потребоваться внешние библиотеки или дополнительные инструменты.

1.5 Обзор фреймворков для разработки серверной части приложения

1.5.1 Веб-фреймворк FastAPI

Для разработки серверной части современных клиент-серверных приложений широко применяются фреймворки, обеспечивающие обработку HTTP-запросов, маршрутизацию, валидацию данных и формирование ответов в удобном для клиента формате. Одним из таких инструментов является FastAPI – современный веб-фреймворк для языка Python, предназначенный для создания API и отличающийся высокой производительностью [8].

Основными преимуществами FastAPI являются: высокая скорость разработки, использование стандартных аннотаций типов Python, поддержка механизма внедрения зависимостей, а также встроенная поддержка автоматической генерации документации API. Наличие автоматически создаваемой документации повышает наглядность структуры API и облегчает интеграцию серверного компонента с клиентским приложением. Кроме того, FastAPI подходит для построения асинхронных приложений, что важно при обработке большого количества одновременных запросов и взаимодействии с внешними сервисами.

Недостатки FastAPI:

– для построения полноценного серверного приложения требуется дополнительная настройка компонентов, связанных с безопасностью, хранением данных и развертыванием;

– часть возможностей реализуется за счет внешних библиотек и подключаемых компонентов;

– при разработке сложного серверного приложения требуется отдельно организовывать структуру модулей и вспомогательных подсистем.

1.5.2 Веб-фреймворк Express

Express – это веб-фреймворк для платформы Node.js, предназначенный для разработки серверных приложений и API. Одной из его основных особенностей является простота построения маршрутов и обработки HTTP-запросов, что позволяет достаточно быстро создавать серверную часть клиент-серверных приложений. За счет гибкой архитектуры Express может использоваться для реализации REST API, обработки запросов от мобильных и веб-клиентов, а также интеграции с различными внешними сервисами [9].

К плюсам Express относятся:

– гибкость жестких ограничений на структуру приложения;

– развитая система middleware;

– удобство создания REST API;

– совместимость с множеством библиотек.

К минусам Express относятся:

– необходимость самостоятельной настройки многих компонентов приложения;

– отсутствие большого количества встроенных средств для валидации данных, безопасности и организации структуры проекта;

– при разработке крупных систем может усложняться сопровождение кода без строгого соблюдения архитектурных решений.

1.5.3 Фреймворк Sping Boot

Spring Boot – это фреймворк для разработки серверных приложений на языке Java, предназначенный для создания самостоятельных и готовых к развертыванию программных решений. Он используется при построении серверной части клиент-серверных систем и позволяет упростить разработку приложений [10].

Важной особенностью Spring Boot является наличие развитой экосистемы и большого количества встроенных инструментов, предназначенных для мониторинга, конфигурирования и сопровождения приложения. Это делает фреймворк удобным для разработки сложных и масштабируемых серверных систем. Кроме того, Spring Boot поддерживает создание REST API, интеграцию с базами данных, организацию безопасности и обработку HTTP-запросов, а также взаимодействие с внешними сервисами и микросервисной архитектурой. Стоит также отметить высокую распространённость данного решения, благодаря чему существует большое количество документации, библиотек.

Недостатки Spring Boot:

– более сложная структура проекта по сравнению с остальными фреймворками;

– больший объем кода и конфигурации при разработке небольших приложений;

– избыточность использования для сравнительно простых серверных решений.

1.6 Обзор баз данных

Для хранения данных в программных средствах могут использоваться реляционные и нереляционные базы данных. Реляционные базы данных предполагают организацию информации в виде взаимосвязанных таблиц и поддерживают выполнение SQL-запросов. Нереляционные базы данных применяются для хранения данных иной структуры и используются в случаях, когда требуется более гибкий подход к их организации. Для разрабатываемого программного средства рассматриваются реляционные базы данных. Среди наиболее распространенных решений можно выделить:

1 PostgreSQL – это объектно-реляционная система управления базами данных с открытым исходным кодом, предназначенная для надежного хранения и обработки сложных наборов данных. Данная СУБД хорошо подходит для клиент-серверных приложений, в которых требуется работа с большим количеством взаимосвязанных сущностей [11].

К преимуществам PostgreSQL относятся: развитые средства работы со взаимосвязанными данными, поддержка расширенного SQL-функционала, гибкость структуры данных, возможность дальнейшего расширения системы.

К недостаткам PostgreSQL относятся: необходимость наличия отдельного серверного процесса, более сложная настройка по сравнению со встраиваемыми решениями.

2 MySQL – реляционная система управления базами данных, широко применяемая при разработке веб- и серверных приложений. Она используется для хранения структурированных данных и поддерживает многопользовательский режим работы [12].

Преимуществами MySQL являются: широкая распространенность, наличие большого количества документации и примеров использования, поддержка SQL и средств работы со структурированными данными, быстрая обработка данных.

Недостатками MySQL являются: меньшая гибкость при работе со сложными взаимосвязанными структурами данных, необходимость более тщательной организации схемы базы данных при расширении функциональности приложения.

3 SQLite – это встраиваемая реляционная база данных, реализованная в виде библиотеки и не требующая отдельного серверного процесса. Данное решение часто применяется в мобильных и локальных приложениях, где необходимо обеспечить хранение данных внутри самого программного средства [13].

К преимуществам SQLite относятся: простота использования, отсутствие необходимости настройки сервера, удобство встраивания в приложение, хранение данных в одном кроссплатформенном файле, высокая производительность при локальных операциях и открытый исходный код.

К недостаткам SQLite относятся: ориентация преимущественно на локальное хранение данных, меньшая применимость в качестве основной базы данных серверной части приложения.

1.7 Обзор шаблонов проектирования

Разработка мобильных приложений на языке программирования Swift предполагает выбор архитектурного подхода, определяющего структуру приложения и взаимодействие его составных частей. Наиболее распространёнными архитектурными паттернами в данном случае являются Модель–представление–контроллер и Модель–Представление–Модель представления [14].

Модель–представление–контроллер (Model–View–Controller, MVC) – это архитектурный паттерн, широко применяемый при разработке программных приложений. Его основная идея заключается в разделении системы на три взаимосвязанных компонента: модель (Model), представление (View) и контроллер (Controller). Такой подход способствует повышению гибкости, упрощает сопровождение программного продукта и делает его более масштабируемым.

Структура паттерна MVC представлена на рисунке 1.4.

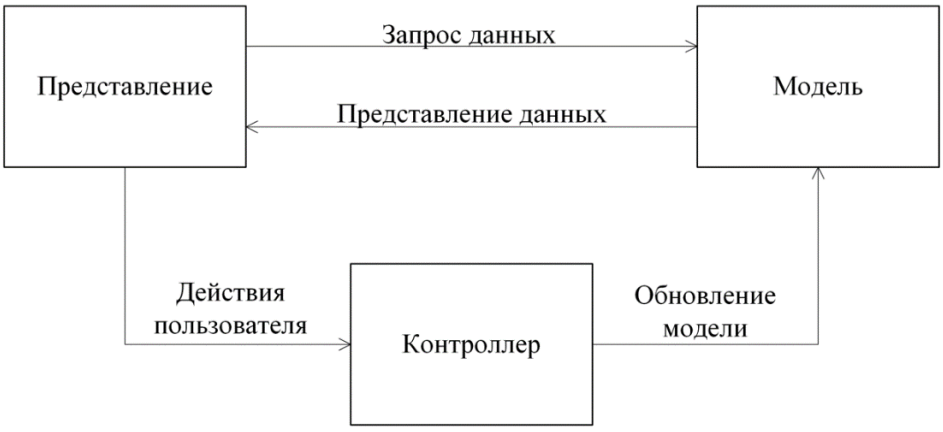


Рисунок 1.4 – Структура паттерна MVC

Модель в шаблоне MVC представляет собой основной компонент, в котором сосредоточена бизнес-логика приложения. Она отвечает за обработку данных, выполнение вычислений, выборку и изменение информации в базе данных. Данные поступают в модель через контроллер, после чего выполняется их необходимая обработка: обращение к базе данных, выполнение вычислительных операций и реализация прикладной логики. Таким образом, модель описывает данные приложения и правила работы с ними [15].

Представление отвечает за отображение данных пользователю. Пользователь взаимодействует с приложением через представление, а выполняемые им действия приводят к обращению к бизнес-логике,

воздействующей на данные модели.

Контроллер является посредником между моделью и представлением. Его основная задача заключается в обработке действий пользователя, поступающих из представления, и передаче соответствующих команд модели. Контроллер организует взаимодействие между компонентами, управляет потоками данных в приложении и определяет, какое представление должно быть сформировано в ответ на действия пользователя. После обработки запроса модель формирует результат и передаёт его контроллеру, который, в свою очередь, выбирает соответствующее представление и возвращает его пользователю.

Паттерн Модель–Представление–Модель представления (Model–View–ViewModel, MVVM) предполагает разделение приложения на три основных компонента: модель, представление и модель представления. Данный подход во многом схож с MVC, однако отличается способом организации взаимодействия между компонентами [16].

Основная идея MVVM заключается в том, что представление связывается с моделью представления посредством механизма привязки данных (data binding). Благодаря этому изменение состояния ViewModel автоматически приводит к обновлению представления, а действия пользователя, выполненные в интерфейсе, могут изменять данные модели представления без необходимости явного вызова методов обновления представления.

Структура паттерна MVVM представлена на рисунке 1.5.



Рисунок 1.5 – Структура паттерна MVVM

Модель в паттерне MVVM, как и в MVC, отвечает за хранение данных, их обработку и реализацию бизнес-логики приложения. В данном компоненте выполняются вычисления, проверка корректности данных, а также взаимодействие с базой данных или другими источниками информации.

Представление, в свою очередь, предназначено для отображения данных пользователю и обработки пользовательского ввода. Однако, в отличие от MVC, представление в MVVM не взаимодействует с контроллером, а связывается непосредственно с моделью представления.

Модель представления является промежуточным звеном между моделью и представлением и по своему назначению частично напоминает контроллер из MVC. Вместе с тем её функции шире, поскольку она не только передаёт данные между компонентами, но и предоставляет представлению свойства и команды, удобные для отображения и взаимодействия с

пользователем. За счёт использования механизма связывания данных изменение состояния ViewModel автоматически отражается в представлении, а действия пользователя, выполненные в интерфейсе, могут приводить к изменению данных модели представления без необходимости явного обновления интерфейса программными средствами.

К преимуществам паттерна MVVM следует отнести чёткое разделение пользовательского интерфейса, логики представления и бизнес-логики приложения, что повышает модульность программного средства, упрощает сопровождение кода и облегчает его дальнейшее расширение. Также к преимущества относится использование механизма привязки данных. Вместе с тем применение MVVM требует более сложной организации структуры приложения и большего объёма исходного кода по сравнению с MVC.

1.8 Обзор фреймворков для разработки клиентской части

Для разработки пользовательского интерфейса приложений для платформы iOS существуют два основных фреймворка: UIKit и SwiftUI.

UIKit является классическим фреймворком для построения пользовательского интерфейса на платформах Apple. Он предоставляет основные объекты, необходимые для отображения содержимого на экране, обработки действий пользователя и управления переходами между экранами приложения [17]. Для UIKit характерен преимущественно императивный подход, при котором разработчик явно задаёт создание, настройку и изменение элементов интерфейса. Кроме того, приложения, разрабатываемые с использованием UIKit, традиционно строятся на основе архитектурного паттерна MVC.

SwiftUI представляет собой более современный фреймворк для разработки пользовательского интерфейса, основанный на декларативном подходе. При его использовании разработчик описывает итоговое состояние представления, а система самостоятельно формирует и обновляет его в зависимости от текущих данных [18]. SwiftUI позволяет задавать иерархию представлений вместе с их зависимостью от состояния приложения, благодаря чему изменение данных автоматически отражается в интерфейсе. При разработке приложений с использованием SwiftUI наиболее целесообразным является применение архитектурного паттерна Model–View–ViewModel. Одним из основных преимуществ SwiftUI является более простой и интуитивный синтаксис для описания пользовательского интерфейса. Кроме того, SwiftUI обладает рядом функций, которых нет в UIKit, таких как автоматические анимации и адаптивный дизайн, что упрощает создание интерфейсов, которые могут адаптироваться к разным размерам экранов.